

The Augmented LLM: The Cell Every Organism Is Built From

Chapter 2, objective 1.3a. One model call that can retrieve, call tools and hold memory. Exhibit E1 decoded in plain words.

One clerk with three drawers. Picture one clerk at one desk with three drawers: a filing cabinet (retrieval), a telephone (tools), and a notebook (memory). The clerk decides for himself when to open which drawer. He is not a department and not a process chart. One competent clerk, augmented. Every pattern in this book is just this clerk, photocopied and arranged.

The taxonomy with three floors

Anthropic's taxonomy stacks three floors. The **augmented LLM** is one model call that can retrieve, call tools and hold memory. **Workflows** are several such calls orchestrated through *predefined code paths*. **Agents** are systems where the model *dynamically directs its own process and tool usage*. Hold on to the two italic phrases. They are the exam's dividing lines and they appear in stems almost word for word.

Floor	Who decides the next step	Everyday picture
Augmented LLM	Nobody has to. One question, one answer, maybe one phone call	A clerk answering one customer
Workflow	Your code, written before any model speaks	An assembly line the clerk stands in
Agent	The model, by looking at each result	A detective choosing the next lead

Exhibit E1: KAVI triages one exception

The book's first exhibit is about 40 lines. One call, one tool, no loop beyond a single round-trip. You do not need to run it. You need to be able to tell its story, line by line, because the agent loop in Part 4 is this same story with a for-loop wrapped around it.

Lines	What happens	In plain words
1 to 4	Imports. DRY_RUN read from the environment. MODEL set to a current alias.	The light switch by the door. Flip DRY_RUN on and the whole building runs on battery: no bill, no network. The free probe always comes before the paid call, the same habit as DRY_RUN in E2 and E5 of the lab.
6 to 12	tools: one entry named get_shipment_status, with a description and a JSON schema that requires consignment_id.	A tool is a promise, not code. It is a phone-book entry: here is my name, here is what I do, to call me give me a consignment id string. The required list says: no id, no call.
14 to 15	fake_status returns a canned record: CUSTOMS_HOLD, three days late.	The canned world. When the building is on battery, someone still has to answer the phone. Your prediction stays something solid to check against.
17 to 20	triage() opens. If DRY_RUN, return a canned verdict and stop. Otherwise create the client.	First move: on battery, hand back the canned verdict. The paid clerk is never disturbed until the free rehearsal has run.
21 to 28	One messages.create: system says reply as JSON in six categories, tools hands over the phone book, messages carries the email as the user turn.	The one paid call. The system prompt is the clerk's standing orders. The tools list is the phone book. The email rides in as the user turn.
29	if msg.stop_reason == tool_use	The hinge of the whole chapter. The response's stop_reason is the clerk's note on top of the file. tool_use means: I picked up the phone, connect me. Anything else means the file is finished.
30 to 32	Find the tool_use block, read its input, call fake_status with it.	Reading the phone request and dialling. In production this line is the real MCP tool from the lab's E1.
33 to 38	A second messages.create with the whole conversation so far replayed, plus one new user turn carrying the tool_result, matched by tool_use_id.	The receipt. The API is stateless, so you replay everything: the email, the clerk's reply, then the result. tool_use_id matches the receipt to the request the way a courier receipt matches a parcel. Forget to replay the assistant turn and the clerk has amnesia.
39 to 41	Return the text. Print the triage of Padma's email.	Padma's email is triaged. One exception, at most two paid calls, no loop. This is the cell.

The habit the book drills: predict before run

Before the battery run, write down what will print and how many paid calls were made. The answer is the canned verdict and zero. Before the live run, write down which branch fires. For Padma's email the model calls the tool, because the email names a consignment and asks where it is, and the clerk cannot know that without picking up the phone. The point is not the answer. The point is having written a prediction that the run can prove wrong.

Sabotage lab: break exactly one variable

Delete the required list from the schema. Now the model may call the tool with an empty input, and fake_status throws a KeyError. The lesson, in the book's words: **the schema is a guardrail you own; the model's compliance is a behaviour you rent**. Put the line back. One variable, one lesson.

Drift: prefilled assistant turns. Older material forces JSON by pre-filling the assistant's reply with an opening brace. Current docs: assistant-message prefill is unsupported on newer models and returns a 400. The current route is structured outputs through output_config.format with a JSON schema, or instructions plus validation in code, as E1 does. I checked this against the

API reference loaded earlier in this session, and it agrees: prefill is removed on the current model family.

How this connects to Domain 3

The clerk's telephone is the MCP tool the lab's E1 served. The phone book is the schema E2 priced at about 193 tokens each. The receipt matched by tool_use_id is the same round-trip the lab's probe made when it called get_shipment_status for KF-2481. Domain 3 showed the phone network. Domain 1 shows the clerk deciding whether to pick up the phone.

The one thing to remember. *The augmented LLM is one call that can retrieve, call tools and hold memory. Its reply carries a stop_reason, and that one field decides whether there is a next move. Everything bigger is this cell, arranged.*